Introduction to Python

02 October 2026 07:21

What is Python?

Python is a high - level, general - purpose interpreted programming language known for its simple and readable syntax.

Python was created by Gudio Van Rossum and was first released in 1991.

Python is widely used in:

- Web development
- Data analysis
- Artificial Intelligence
- Machine learning
- Automation
- Scientific Computing
- Desktop applications
- Game Development
- Cybersecurity
- Database programming
- Education

Simple Python Code:

```
print("Hello world")
```

Output:

```
Hello world
```

The print() function displays information on the screen.

Why learn Python?

Python is popular because its syntax is relatively easy to understand.

Python requires considerably less boilerplate, then C++ or Java.

Major Advantage

1. **Easy to learn.**
2. **Easy to read.**
3. **Open source.**
4. **Cross - platform.**
5. **Large standard library.**
6. **Large ecosystem of 3rd - party libraries.**
7. **Supports multiple programming paradigms.**
8. **Suitable for beginners as well as professionals.**

Features of Python

1. **Simple**
2. **High level language**
3. **Dynamically typed language:** Python is dynamically tied. We do not normally have to specify a variable's data type when creating it.

```
PS C:\Users\bluej\Documents\Students\KapishKhemkaLMB2628\ForGit\Coding\PythonProjects> py
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Python Interpreter or shell

In the above screen, the symbol >>> is the Python prompt, which indicates that the interpreter is ready to take instructions. We can type commands or statements on this prompt to execute them using a Python Interpreter.

Execution Modes

There are two ways to use the Python interpreter:

1. Interactive mode.
2. Script mode

Interactive mode

To work in the interactive mode, we can simply type a Python statement on the >>> prompt directly. As soon as we press enter, the interpreter executes the statement and displays the results.

```
PS C:\Users\bluej\Documents\Students\KapishKhemkaLMB2628\ForGit\Coding\PythonProjects> py
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3
5
>>> 2 * 3
6
>>> 2 ** 3
8
>>> 9 ** 0.5
3.0
>>> name = input('Enter name: ')
Enter name: Shiv
>>> name
'Shiv'
>>>
```

Python interpreter in interactive mode

Working in the interactive mode is convenient for testing a single line code for instant execution. But in the interactive mode, we cannot save the statements for future use and we have to retype the statements to run them again.

Script Mode

In the script mode, we can write up the Python program in a file, save it and then user interpreter to execute it. Python scripts are saved as file where file name has extension ".py".

Python Keywords

Keywords are reserved words. Each keyword has a specific meaning to the

Python interpreter, and we can use the keyword in our program only for the purpose which for which it has been defined.

```
import keyword
print('List of keywords')
klist = keyword.kwlist
print(klist)
print(f'Number of keywords in Python: {len(klist)}')
```

Output:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try',
'while', 'with', 'yield']
Number of keywords in Python: 35
```

Identifiers

In programming languages, identifiers are names used to identify a variable, function, or other entities in a program. The rules for naming an identifier in Python are as follows:

- The name should begin with. An upper case or lower case alphabet or an (_)sign. This may be followed by any combination of character A-Z, a-z, 0-9 or underscore(_). Thus an identifier cannot start with Digit.
- It can be of any length. (However, it is preferred to keep it short and meaningful.)
- It should not be a keyword or reserved word.
- We cannot use special symbols like !, @, #, $, %, ^, &, * etc. in identifier.

```
length = float(input('Enter length: '))
breadth = float(input('Enter breadth: '))
area = length * breadth
print(f'Area: {area}')
```

Output:

```
Enter length: 2.5
Enter breadth: 1.5
Area: 3.75
```

Comments

Comments are used to add a remark or a note in the source code. Comments are not executed by interpreter. They are added with the purpose of making the source code easier for humans to understand. They are used primarily to document the meaning and purpose of source code and its input and output requirements, so that we can remember later how it functions and

how to use it. For large and complex software, it may require programmers to work in teams, and sometimes a program written by one programmer is required to be used or maintained by another programmer. In such situation, documentation in the form of comments are needed to understand the working of the program.

And in Python a comment starts with hash sign (#). Everything following the # till the end of that line is treated as a comment and the interpreter simply ignores it while executing the statement.

multi-line comment, is written using triple quote (''' '''), (""" """)

```
'''
    In this program will take the input in length and breadth.
    Calculate area and store it in 'area' variable
'''

length = float(input('Enter length: '))
breadth = float(input('Enter breadth: '))
# Calculating area of rectangle.
#
#
area = length * breadth
print(f'Area: {area}')
```

Everything is an Object

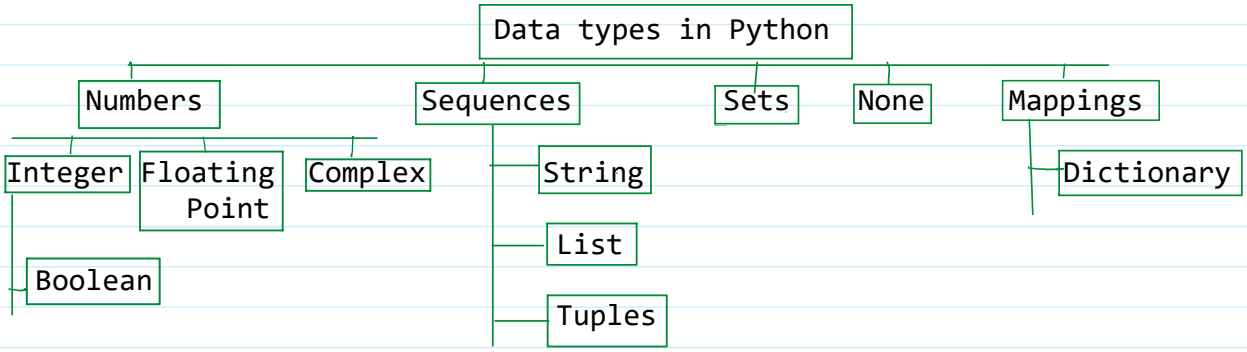
Python treats every value or data item, whether numeric, string or other type. As an object in the sense that it can be assigned to some variable or can be passed to a function as an argument.

Every object in Python is assigned a unique identity (ID) Which remains the same for the lifetime of that object. This ID is akin to the memory address of the object. The function id() returns the identity of an object.

```
>>> id(s3)
2495181110864
>>> id(s1)
2495180842736
>>> id(s2)
2495180843504
>>> d = 10
>>> type(d)
<class 'int'>
>>> id(d)
140713076815256
>>> d = 'Jack'
>>> type(d)
<class 'str'>
```

Data types

Every value belongs to a specific data type in Python. Data type identifies the type of data values a variable can hold and the operations that can be performed on that data.



Different data types in Python

Number

Number data type stores numerical values only. It is further classified into three different types: int, float and complex.

Numeric data types

Type/class	Description	Examples
int	Integer numbers	-12, -3, 0, 125, 18
float	Real or floating point numbers	-2.05, 4.0, 14.23
complex	Complex numbers	(3 + 5j), 0j, (2 - 2j)

Boolean data type (bool) is a subtype of Integer. It is unique data type consisting of two constants true and false. Boolean *True* value is a non-zero, non-null and non-empty. Boolean *False* is the value zero.

```
>>> b = True
>>> type(b)
<class 'bool'>
```

Sequence

A Python sequence is an ordered collection of items, where each item is indexed by an integer. The three types of sequences data type available in Python are strings, list and tuples.

Set

Set is an unordered collection of items separated by and. The items are enclosed in curly brackets {} A set is similar to list except that it cannot have duplicate entries. Once created, elements of a set cannot be changed.

None

None is a special data type with a single value. It is used to signify the

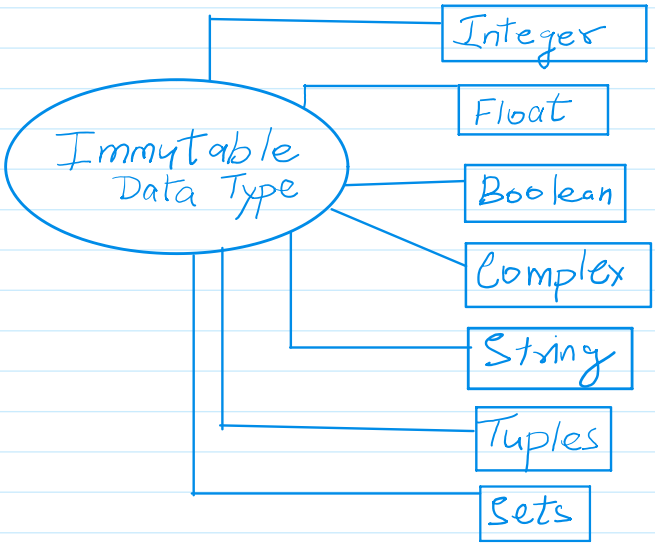
absence of values in a situation. None supports no special operation. And it is neither False nor 0 (zero).

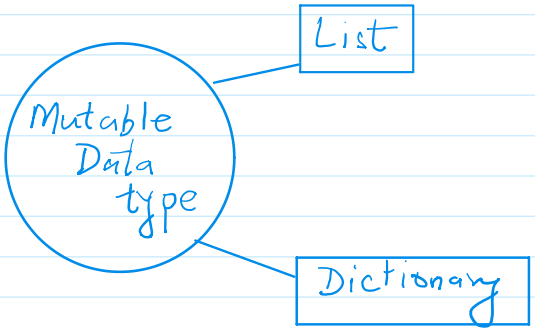
```
>>> v = None
>>> type(v)
<class 'NoneType'>
>>> v
```

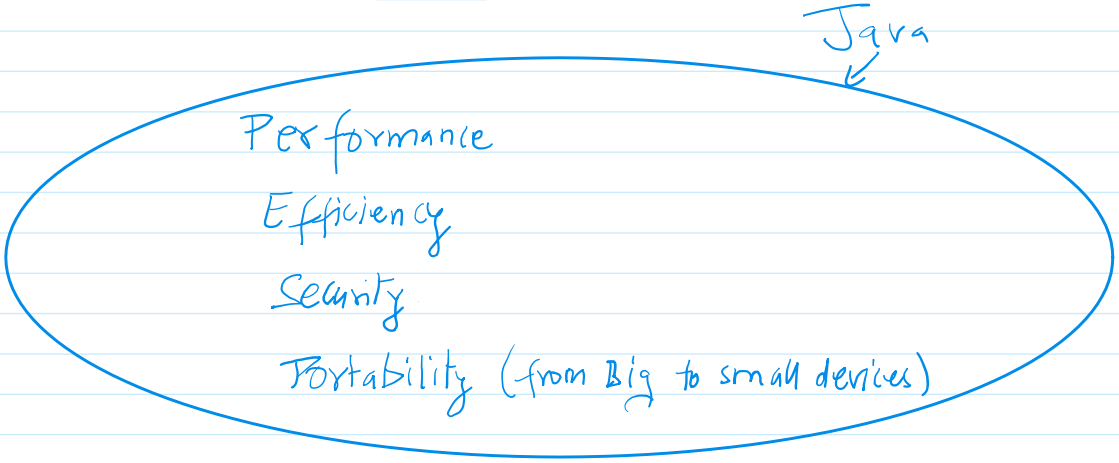
Mapping
Mapping is an unordered data type in Python. Currently there is only one standard mapping data type in Python called a dictionary.

Mutable and Immutable Data Types
Sometimes we may require to change or update the values of a certain variable used in the program. However, for certain data types, Python does not allow us to change the value once the variable of that data type has been created and assigned values.

Variables whose values can be changed after they are created and assigned are called mutable. Variable whose values cannot be changed after they are created and assigned are called immutable. When an attempt is made to update the value of an immutable variable, the old variable is destroyed and the new variable is created by the same name in memory.







Operators

Symbols use to represent operations.

1. Arithmetic operator: +, -, *, /, //(floor division), %(remainder), **(power/index returns exponential value)
2. Relational operators: ==, !=, >, <, >=, <=
3. Assignment operator: =, +=, -=, *=, /=, //=, %=, **=
4. Logical Operators: and, or, not

   The logical operator evaluates to either True or False based on the logical operands on either side. Every value is logically either true or false. By default all values are True except None, False, 0 (Zero) empty collections, "",(), [], {} And few other special values.

5. Identity Operators: Identity operators are used to determine whether the value of a variable is of certain type or not. Identity operators can also be used to determine whether two variables are referring to the same object or not. There are two identity operators.

Operator	Description	Example

is	Evaluates True if the variables on either side of the operator points towards the same memory location, and False otherwise.	**>>> 9 ** 0.5
		3.0
		>>> num1 = 5
		>>> type(num1) is int
		True
		>>> num2 = num1
		>>> id(num1)
		140716735624440
		>>> id(num2)
		140716735624440
		>>> num1 is num2
		True
		>>> num3 = 'Hello'
		>>> num1 is num3
		False**

is not	Evaluates to false if the variable on either side of the operator points to the same memory location, and true otherwise.	**>>> type(num3)
		<class 'str'>
		>>> type(num1)
		<class 'int'>
		>>> type(num2)
		<class 'int'>
		>>>
		>>> num1 is not num2
		False
		>>> num1 is not num3
		True**

Membership operator

Membership operators are used to check if a value is a member of the given sequence or not.

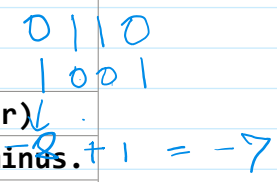
Operator	Description	Example

in	Returns true if the variable/ value is found in the specified sequence and false otherwise.	>>> a = [1,2,3]
		>>> 2 in a
		True
		>>> 5 in a
		False

not in	Returns True if the variable / value is not found in the specified sequence, and False otherwise.	>>> a
		[1, 2, 3]
		>>> 5 not in a
		True
		>>> 3 not in a
		False

Precedence of operator

Order of precedence	Operators	Description
1	**	Exponentiation (raised to the power)
2	~, +, -	Complement, Unary plus and unary minus.
3	*, /, //, %	Multiply, divide, modulo and floor division.
4	+, -	Addition and subtraction.
5	<=, <, >, >=	Relational operators.
6	==, !=	Equality operators.
7	=, %=, /=, //=, -=, +=,*=, **=	Assignment operators.
8	is, is not	Identity operators.
9	in, not in	Membership operators.
10	not, or, and	Logical operators.



Note:

a. Parenthesis can be used to override the precedence of operators. The expression within () is evaluated first.
b. For operators with equal precedence, the expression is evaluated from left to right.

```
print(value[,…, sep = ' ', end = '\n)
```

Code

```
print()
v1, v2, v3 = 10, 25, 30
print(v1, v2, v3, sep='$')
print(v1, v2, v3, end='#')
```

```
    print()
    print(v1, v2, v3, sep='kapish', end='#')

Output:
    10$25$30
    10 25 30#
    10kapish25kapish30#
```